

CargoBot — EventBus Tutorial

QR kod algılamadan PLC bildirimine, tek bir event'in yolculuğu

CargoBot

2026

İçindekiler

Senaryo	1
1. Event tanımı — kendi modülünün içinde	1
2. Yayınlama — kameradan gelen frame'i işleyen adapter	2
3. Abone olma — mission handler	2
4. Bağlantı — wiring tek listede	3
5. Çalışırken akış	3
6. Uçtan uca canlı örnek	3
7. Yeni event eklemek istediğinde 3 adım	4
Pattern özeti	4

Senaryo

Kameradan bir QR kod okunuyor. Mission bu QR'ı bekliyorsa faz değiştiriyor, yükü "alındı" işaretliyor, fleet I/O katmanı PLC'ye TCP üzerinden "yük alındı" mesajı yolluyor.

Hiçbir modül diğerini import etmiyor. Aralarındaki tek köprü: event bus.

1. Event tanımı — kendi modülünün içinde

Event sınıfı kendi context'inin klasöründe yaşar. QrCodeDetected perception'da, MissionPhaseChanged mission'da. Tek dosyada toplamıyoruz.

src/domain/perception/events.py:

```
from dataclasses import dataclass
from domain._shared.events import DomainEvent
from domain._shared.value_objects import Pose
```

```
@dataclass(frozen=True, kw_only=True)
class QrCodeDetected(DomainEvent):
    qr_id: str
```

```
pose_camera: Pose
confidence: float
```

DomainEvent base’i event_id, occurred_at, correlation_id alanlarını otomatik veriyor. frozen=True çünkü event immutable — yayınlandıktan sonra değişmez.

2. Yayınlama — kameradan gelen frame’i işleyen adapter

src/infrastructure/camera/qr_publisher.py:

```
import cv2
from pyzbar.pyzbar import decode
from domain._shared.value_objects import Pose
from domain.perception.events import QrCodeDetected
from eventbus import AsyncEventBus

class QrPublisher:
    def __init__(self, bus: AsyncEventBus):
        self._bus = bus

    async def on_frame(self, frame):
        for code in decode(frame):
            event = QrCodeDetected(
                aggregate_id="camera-front",
                qr_id=code.data.decode(),
                pose_camera=Pose(x=0.0, y=0.0, theta=0.0), # solvePnP buraya
                confidence=1.0,
            )
            await self._bus.publish(event)
```

Adapter event objesini doldurup bus.publish(event) çağırıyor. Kimin dinlediğini bilmiyor — sadece “şu olay oldu” diye duyuruyor.

3. Abone olma — mission handler

src/application/handlers/mission_handlers.py:

```
def on_qr_detected(bus):
    async def handle(event):
        m = _get()
        if m is None:
            return
        if m.phase == MissionPhase.APPROACHING_LOAD and event.qr_id == m.pickup_no:
            m.transition(MissionPhase.PICKING)
            m.mark_picked(event.qr_id)
            await _flush(bus, m)
    return handle
```

Factory deseni: dış bağımlılıkları (burada bus) closure’a alıp asıl handler’ı dönüyor. wiring.py bu handler’ı toplar ve bus’a register eder.

_flush(bus, m) agregada birikmiş event’leri toplayıp bus’a yayımlar. Aggregate doğrudan bus’a yayın yapmaz — domain saf Python kalsın diye event biriktiriyor, handler aktarıyor.

4. Bağlantı — wiring tek listede

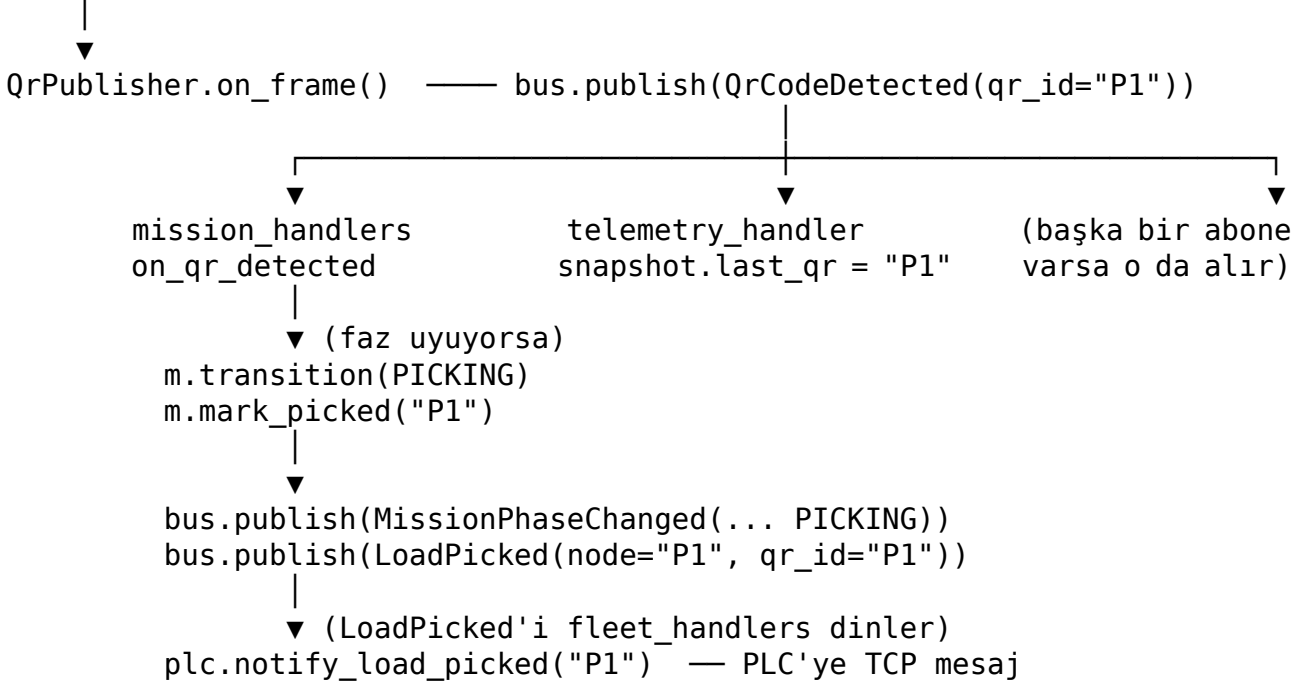
src/application/wiring.py içinde tek satır:

```
bus.subscribe(QrCodeDetected, mission_handlers.on_qr_detected(bus))
```

Tüm pub/sub bağlantıları aynı dosyada toplanır. C++'taki connection manager'ın işi bu. Decorator tabanlı otomatik kayıt yok — okurken nereye bağlanıldığı tek bakışta görünür.

5. Çalışırken akış

kamera frame



6. Uçtan uca canlı örnek

```
async def demo():
    app = App.build(plc=MockPlc(), motor=MockMotor())
    wire(app.bus, app.deps)

    # önce bir atama gelir, mission MOVING_TO_PICKUP'a ulaşır
    await app.bus.publish(PickAssignmentReceived(
        aggregate_id="plc", pickup_node="P1", dropoff_node="D1"
    ))
    await app.bus.publish(WaypointReached(
        aggregate_id="nav", waypoint_id="p1-pre"
    ))
    # mission şimdi APPROACHING_LOAD

    # kamera P1 QR'ı okudu
    await app.bus.publish(QrCodeDetected(
        aggregate_id="camera",
        qr_id="P1",
        pose_camera=Pose(0, 0, 0),
```

```
confidence=0.97,  
) )
```

Tek bir `QrCodeDetected` yayını ardından mission faz değiştiriyor, PLC'ye "yük alındı" mesajı gidiyor. Hiçbir modül diğerini import etmedi.

7. Yeni event eklemek istediğinde 3 adım

1. İlgili context'in `events.py`'sine sınıfı koy.
2. Kaynağında (adapter veya handler) `bus.publish(...)` çağır.
3. `wiring.py`'a bir satır `bus.subscribe(YeniEvent, handler)` ekle.

Bus'ı genişletmek, base class değiştirmek, registry'ye dokunmak yok. Sadece bu üç dosya.

Pattern özeti

Soru	Cevap
Event nerede tanımlı?	Kendi bounded context'inin <code>events.py</code> 'sinde
Kim publish ediyor?	Olayın gerçekten kaynaklandığı yer (adapter veya handler)
Kim subscribe oluyor?	Olayı duymak isteyen handler
Bağlantı listesi nerede?	<code>application/wiring.py</code> — tek dosya
Aggregate bus'ı biliyor mu?	Hayır. Event biriktiriyor, handler aktarıyor
Yeni handler nasıl eklenir?	Handler factory yaz + <code>wiring.py</code> 'a bir satır